Binary Search Tree Data Structure

1. Introduction to Binary Search Tree

A Binary Search Tree (BST) is a hierarchical data structure that maintains its elements in sorted order. It is a binary tree where each node has at most two children, referred to as the left child and right child.

Key Properties of BST:

- **Left Subtree Property**: All values in the left subtree of a node are less than the node's value
- **Right Subtree Property**: All values in the right subtree of a node are greater than the node's value
- **Recursive Property**: Both left and right subtrees are also binary search trees

Basic Node Structure:

```java
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;

    public TreeNode(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}
```

Example BST:

```
        50
       /  \
     30    70
    /  \   /  \
  20  40 60  80
```

In this example:

- All nodes in the left subtree (20, 30, 40) are less than 50
- All nodes in the right subtree (60, 70, 80) are greater than 50

2. Application and Usage

Binary Search Trees are widely used in computer science due to their efficient searching, insertion, and deletion operations.

Common Applications:

Database Indexing: BSTs help maintain sorted indexes for faster data retrieval

```java
// Example: Student database sorted by ID
class Student {
    int studentId;
    String name;
    // BST maintains students sorted by ID
}
```

Expression Parsing: Used in compilers to parse mathematical expressions

```java
// Example: Expression tree for (a + b) * c
//     *
//    / \
//   +   c
//  / \
// a   b
```

File Systems: Directory structures often use tree-like organization

Priority Queues: BSTs can implement priority queues efficiently

Autocomplete Systems: Search engines use BSTs for word suggestions

Time Complexity Benefits:

- **Search**: O(log n) average case
- **Insertion**: O(log n) average case
- **Deletion**: O(log n) average case
- **Traversal**: O(n) for all elements

3. Representation of Binary Search Tree and Their Construction

BST Class Implementation:

java

```java
class BinarySearchTree {
    TreeNode root;

    public BinarySearchTree() {
        this.root = null;
    }

    // Insert method
    public void insert(int data) {
        root = insertRecursive(root, data);
    }

    private TreeNode insertRecursive(TreeNode node, int data) {
        // Base case: create new node
        if (node == null) {
            return new TreeNode(data);
        }

        // Recursive case: choose left or right subtree
        if (data < node.data) {
            node.left = insertRecursive(node.left, data);
        } else if (data > node.data) {
            node.right = insertRecursive(node.right, data);
        }

        return node;
    }

    // Search method
    public boolean search(int data) {
        return searchRecursive(root, data);
    }

    private boolean searchRecursive(TreeNode node, int data) {
        // Base case: not found
        if (node == null) {
            return false;
        }

        // Base case: found
        if (node.data == data) {
            return true;
        }
```

```java
        // Recursive case: search left or right
        if (data < node.data) {
            return searchRecursive(node.left, data);
        } else {
            return searchRecursive(node.right, data);
        }
    }
}
```

Construction Example:

```java
public class BSTExample {
    public static void main(String[] args) {
        BinarySearchTree bst = new BinarySearchTree();

        // Insert elements
        bst.insert(50);
        bst.insert(30);
        bst.insert(70);
        bst.insert(20);
        bst.insert(40);
        bst.insert(60);
        bst.insert(80);

        // Search for elements
        System.out.println("Search 40: " + bst.search(40)); // true
        System.out.println("Search 90: " + bst.search(90)); // false
    }
}
```

Tree Traversal Methods:

```java
// Inorder traversal (Left, Root, Right) - gives sorted order
public void inorderTraversal(TreeNode node) {
    if (node != null) {
        inorderTraversal(node.left);
        System.out.print(node.data + " ");
        inorderTraversal(node.right);
    }
}

// Preorder traversal (Root, Left, Right)
public void preorderTraversal(TreeNode node) {
    if (node != null) {
        System.out.print(node.data + " ");
        preorderTraversal(node.left);
        preorderTraversal(node.right);
    }
}

// Postorder traversal (Left, Right, Root)
public void postorderTraversal(TreeNode node) {
    if (node != null) {
        postorderTraversal(node.left);
        postorderTraversal(node.right);
        System.out.print(node.data + " ");
    }
}
```

4. Size, Sum, Maximum and Height of a Binary Search Tree

Size of BST (Total Number of Nodes):

```java
public int size(TreeNode node) {
    if (node == null) {
        return 0;
    }
    return 1 + size(node.left) + size(node.right);
}

// Usage example:
// int totalNodes = bst.size(bst.root);
```

Sum of All Nodes:

```java
public int sum(TreeNode node) {
    if (node == null) {
        return 0;
    }
    return node.data + sum(node.left) + sum(node.right);
}

// Usage example:
// int totalSum = bst.sum(bst.root);
```

Maximum Value in BST:

```java
public int findMax(TreeNode node) {
    if (node == null) {
        throw new IllegalArgumentException("Tree is empty");
    }

    // In BST, maximum is the rightmost node
    while (node.right != null) {
        node = node.right;
    }
    return node.data;
}

// Recursive approach:
public int findMaxRecursive(TreeNode node) {
    if (node == null) {
        throw new IllegalArgumentException("Tree is empty");
    }

    if (node.right == null) {
        return node.data;
    }
    return findMaxRecursive(node.right);
}
```

Minimum Value in BST:

```java
public int findMin(TreeNode node) {
    if (node == null) {
        throw new IllegalArgumentException("Tree is empty");
    }

    // In BST, minimum is the leftmost node
    while (node.left != null) {
        node = node.left;
    }
    return node.data;
}
```

Height of BST:

```java
public int height(TreeNode node) {
    if (node == null) {
        return -1; // Height of empty tree is -1
    }

    int leftHeight = height(node.left);
    int rightHeight = height(node.right);

    return 1 + Math.max(leftHeight, rightHeight);
}

// Alternative: Height as number of nodes in longest path
public int heightAsNodes(TreeNode node) {
    if (node == null) {
        return 0;
    }

    int leftHeight = heightAsNodes(node.left);
    int rightHeight = heightAsNodes(node.right);

    return 1 + Math.max(leftHeight, rightHeight);
}
```

Complete Example with All Operations:

```java
public class BSTOperations {
    public static void main(String[] args) {
        BinarySearchTree bst = new BinarySearchTree();

        // Build the tree
        int[] values = {50, 30, 70, 20, 40, 60, 80};
        for (int value : values) {
            bst.insert(value);
        }

        // Calculate properties
        System.out.println("Size: " + bst.size(bst.root));
        System.out.println("Sum: " + bst.sum(bst.root));
        System.out.println("Maximum: " + bst.findMax(bst.root));
        System.out.println("Minimum: " + bst.findMin(bst.root));
        System.out.println("Height: " + bst.height(bst.root));
    }
}
```

Quiz Questions

Question 1: What is the key property that makes a binary tree a Binary Search Tree? a) All nodes have exactly two children b) Left child values are less than parent, right child values are greater than parent c) The tree is always balanced d) All leaf nodes are at the same level

Answer: b) Left child values are less than parent, right child values are greater than parent

Question 2: What is the time complexity of searching for an element in a balanced BST? a) O(1) b) O(log n) c) O(n) d) $O(n^2)$

Answer: b) O(log n)

Question 3: Which traversal method of a BST gives elements in sorted order? a) Preorder b) Postorder c) Inorder d) Level order

Answer: c) Inorder

Question 4: In a BST, where is the maximum value located? a) Root node b) Leftmost node c) Rightmost node d) Any leaf node

Answer: c) Rightmost node

Question 5: If a BST has height h, what is the maximum number of nodes it can contain? a) h b) 2h c) 2^h d) 2^(h+1) - 1

Answer: d) 2^(h+1) - 1

Practice Coding Questions

Question 1: Write a method to find the second largest element in a Binary Search Tree.

Question 2: Implement a method to check if a given binary tree is a valid Binary Search Tree.

Question 3: Write a method to find the Lowest Common Ancestor (LCA) of two nodes in a BST.

Question 4: Create a method that converts a sorted array into a balanced Binary Search Tree.

Question 5: Implement a method to delete a node from a Binary Search Tree while maintaining the BST property.